NEW HORIZON COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Academic Year: 2026–2027

ALTERNATIVE ASSESSMENT TEST - 1 (AAT-1)

PORTFOLIO-DRIVEN PRACTICAL PROJECT REPORT (15 MARKS)

A Practical Technical Report Submitted for the Course

FULL STACK TECHNOLOGIES - 22CSE71

PROJECT TITLE

TASKFLOW – FULL STACK TASK TRACKER

STUDENT & SUBMISSION DETAILS

Student Name	**Tharun Sivakumar**
University Seat Number (USN)	1NH23CS278
Semester / Section	7E
Course / Subject	Full Stack Technologies - 22CSE71
Assessment Title	AAT-1: Portfolio-Driven Project Development (15 Marks)
Project Title	TaskFlow – Full Stack Task Tracker
GitHub Repository Link	https://github.com/Veins19/FST_AAT1
Date of Submission	__ / __ / 2026 (Deadline: On or before 16-10-2026)

EVALUATION SCHEME – AAT-1 (FOR FACULTY USE)

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Project relevance and implementation	5	
2	GitHub repository and documentation (README.md, structure)	4	
3	Functionality and output	3	
4	Learning demonstrated	2	
5	Timely hard-copy submission	1	
TOTAL MARKS		**15**	

1. PROJECT TITLE

"TASKFLOW – FULL STACK TASK TRACKER"

A REST API-Driven Task Management Web Application with Priority Tracking, Filtering, Search and Live Statistics

GitHub Portfolio URL: https://github.com/Veins19/FST_AAT1

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Students and working professionals handle many assignments, project milestones and personal commitments at the same time, each with a different priority and deadline. Tracking these in paper lists, chat messages or scattered notes makes it hard to see what is still pending, which items are overdue and what has already been completed. Existing task tools are often heavy, account-based or cluttered with features that a single user does not need. Therefore, there is a need for a lightweight, self-hosted full-stack `TaskFlow` application that stores tasks centrally behind a validated REST API and presents them through a clean, responsive browser interface with instant filtering, search and progress statistics.

2.2 Project Objectives

- **RESTful Backend Service:** To design and implement an Express.js API (`/api/tasks`, `/api/stats`) supporting create, read, update and delete operations with correct HTTP status codes (201, 204, 400, 404).
- **Responsive Frontend Client:** To build a single-page interface in HTML, CSS and vanilla JavaScript that consumes the API using the Fetch API and re-renders after every change.
- **Server-Side Validation & Safe Rendering:** To validate titles, priorities and due dates on the server and to HTML-escape user text so that stored content cannot inject scripts (XSS).
- **Productivity Features:** To provide priority badges, due dates with overdue highlighting, All / Active / Done filtering, title search and live Total / Active / Done counters.
- **Testing & Portfolio Publishing:** To verify the API with automated tests and publish the complete source code with a structured `README.md` and screenshots on GitHub.

3. TECHNOLOGIES / TOOLS USED

Module / Layer	Technology / Tool	Purpose in Project
Frontend `public/`	HTML5, CSS3, JavaScript (ES2020), Fetch API	Interactive task interface with stats cards, filter tabs, search box, priority badges and inline validation messages.
Backend Server `app.js`	Node.js 22, Express.js 4	REST API routes, input validation, JSON body parsing and static file serving.
Data Storage `data/tasks.json`	JSON file persistence (Node `fs`)	Lightweight storage that survives server restarts; excluded from Git via `.gitignore`.
Testing `test/`	`node:test`, `assert`	Integration tests that start the app on a random port with a temporary data file.
Version Control	Git, GitHub	Source hosting and public portfolio showcase with README documentation.

4. PROJECT DESCRIPTION

4.1 System Overview and Architecture

`TaskFlow` is an end-to-end full-stack web application in which a browser client communicates with an Express server over JSON. A user can add a task with a title, priority (Low / Medium / High) and an optional due date, mark it done with a checkbox, delete it, filter the list by status, and search by title. Tasks that are past their due date and not completed are highlighted in red. As structured in the public GitHub repository (`FST_AAT1`), the project is organised into three cooperating parts:

1. **Frontend Task Interface (`public/`):** `index.html`, `style.css` and `app.js` render the statistics cards (total, active, done), the add-task form, the filter tabs, the search box and the task list. All task titles are escaped before being inserted into the DOM.
2. **Backend API (`app.js, server.js`):** `createApp(dataFile)` builds the Express application and accepts the storage path as a parameter, so tests can use a temporary file. A `validate()` helper normalises and checks every request body before it is saved.
3. **Automated Tests (`test/api.test.js`):** Two integration tests exercise the full create → update → filter → delete life-cycle and all invalid-input cases against a live server instance.

4.2 Repository Module Breakdown & Data Model

Table 4.1 lists the repository layout and the task entity stored by the system.

Repository Directory / Entity	Key Files & Components	Functional Responsibility
`app.js`, `server.js`	Express app factory, routes, `validate()`, entry point	Implements the REST API, validation rules and JSON file persistence; `server.js` starts the server on port 3000.
`public/`	`index.html`, `style.css`, `app.js`	Renders stats, form, tabs, search and task list; calls the API using `fetch`.
`test/`	`api.test.js`	Verifies CRUD behaviour, filters, statistics and error responses (400 / 404).
`docs/screenshots/`	Six PNG screenshots	Output evidence used in `README.md` and in this report.
Task Record	`id`, `title`, `priority`, `dueDate`, `done`, `createdAt`	One task. Title ≤ 120 characters; priority ∈ {low, medium, high}; dueDate is empty or a valid date.

4.3 REST API Summary

Method	Endpoint	Description
GET	`/api/tasks?status=&q=`	List tasks; optional filter by `active` / `done` and title search.
POST	`/api/tasks`	Create a task (201) or return a validation error (400).
PUT	`/api/tasks/:id`	Update fields such as `done`; 404 if the task does not exist.
DELETE	`/api/tasks/:id`	Delete a task (204); 404 if it does not exist.
GET	`/api/stats`	Returns `{ total, active, done }` for the statistics cards.

5. IMPLEMENTATION / METHODOLOGY

The implementation of **TaskFlow** follows a five-stage engineering workflow that moves from requirement analysis to a published, tested portfolio project:

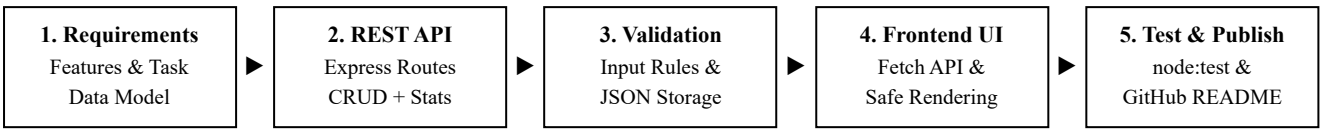


Figure 5.1: Five-Stage Full-Stack Development, Validation, Testing and Publishing Pipeline

5.1 Validation Rules & Statistics Formulation

Every request body is passed through `validate()` before it is stored. A title is trimmed and must satisfy **1 ≤ length ≤ 120**; the priority must belong to the set **{low, medium, high}**; and a non-empty due date must be parseable by `Date.parse`. Invalid input returns HTTP 400 with a readable message that the interface shows under the form. For partial updates (`PUT`) only the supplied fields are validated. The dashboard counters are derived from the stored tasks as:

$$\textbf{Active} = \textbf{Total} - \textbf{Done} \quad \text{and} \quad \textbf{Overdue}(t) = (\neg t.\text{done}) \wedge (t.\text{dueDate} < \text{today})$$

In the demonstration data set of 5 tasks (1 completed), the API returns **total = 5, active = 4, done = 1**, and the task due on 2026-10-16 is rendered as **overdue**.

5.2 Step-by-Step Execution & Deployment Methodology

1. **Clone the Repository:** Execute `git clone https://github.com/Veins19/FST_AAT1.git` and navigate into the project folder (`cd FST_AAT1`).
2. **Install Dependencies:** Run `npm install` to download Express and generate the `node_modules` folder (Node.js 18 or higher is required).
3. **Start the Server:** Run `npm start`. The console prints `TaskFlow running at http://localhost:3000`.
4. **Open the Application:** Visit `http://localhost:3000` in any browser to add, complete, filter, search and delete tasks. Data is saved to `data/tasks.json`.
5. **Run the Automated Tests:** Execute `npm test`; the expected result is *2 tests passed, 0 failed*.

6. IMPORTANT CODE SNIPPETS

The following core snippets from the `FST_AAT1` repository illustrate server-side validation, the create-task route, XSS-safe rendering on the client and the automated API test.

6.1 Server-Side Input Validation (`app.js`)

```
  const validate = (body, partial = false) => {
    const out = {};
    if (!partial || body.title !== undefined) {
      const title = String(body.title ?? '').trim();
      if (!title) return { error: 'Title is required' };
      if (title.length > 120) return { error: 'Title must be 120 characters or fewer' };
      out.title = title;
    }
    if (body.priority !== undefined) {
      if (!PRIORITIES.includes(body.priority)) return { error: 'Priority must be low, medium or high' };
      out.priority = body.priority;
    }
    if (body.dueDate !== undefined) {
      if (body.dueDate !== '' && Number.isNaN(Date.parse(body.dueDate))) return { error: 'Invalid due date' };
      out.dueDate = body.dueDate;
    }
    if (body.done !== undefined) out.done = Boolean(body.done);
    return { value: out };
  };
```

6.2 Create-Task REST Route (`app.js`)

```
  app.post('/api/tasks', (req, res) => {
    const { error, value } = validate(req.body);
    if (error) return res.status(400).json({ error });
    const tasks = load();
    const task = {
      id: Date.now().toString(36) + Math.random().toString(36).slice(2, 6),
      priority: 'medium',
      dueDate: '',
      done: false,
      createdAt: new Date().toISOString(),
      ...value,
    };
    tasks.unshift(task);
    save(tasks);
    res.status(201).json(task);
  });
```

6.3 XSS-Safe Rendering (`public/app.js`)

```
// Escape user text before inserting into HTML (prevents XSS).
const esc = (s) => s.replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }
[c]));
```

6.4 Integration Test – CRUD Life-Cycle (`test/api.test.js`)

```
test('create, list, update, filter and delete a task', async () => {
  let res = await call('/tasks', 'POST', { title: 'Write report', priority: 'high' });
  assert.strictEqual(res.status, 201);
  const task = await res.json();
  assert.strictEqual(task.priority, 'high');
  assert.strictEqual(task.done, false);

  res = await call(`/tasks/${task.id}`, 'PUT', { done: true });
  assert.strictEqual((await res.json()).done, true);

  assert.strictEqual((await (await call('/tasks?status=done')).json()).length, 1);
  assert.strictEqual((await (await call('/tasks?status=active')).json()).length, 0);
  assert.deepStrictEqual(await (await call('/stats')).json(), { total: 1, done: 1, active: 0 });

  assert.strictEqual((await call(`/tasks/${task.id}`, 'DELETE')).status, 204);
  assert.strictEqual((await call(`/tasks/${task.id}`, 'DELETE')).status, 404);
```

7. OUTPUT SCREENSHOTS

The operational output of **TaskFlow** is presented in Figures 7.1 and 7.2. Figure 7.1 shows the main screen with live statistics, the add-task form, the filter tabs and the task list.

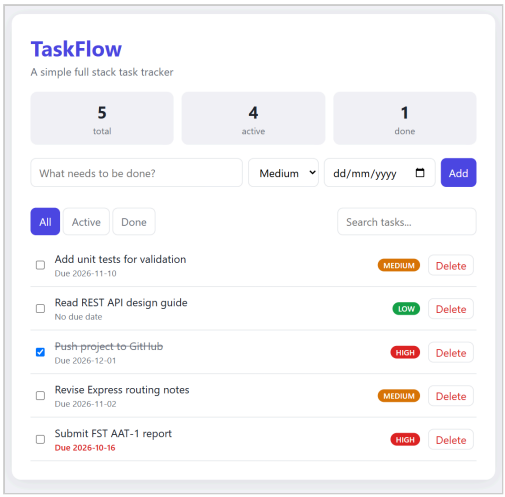


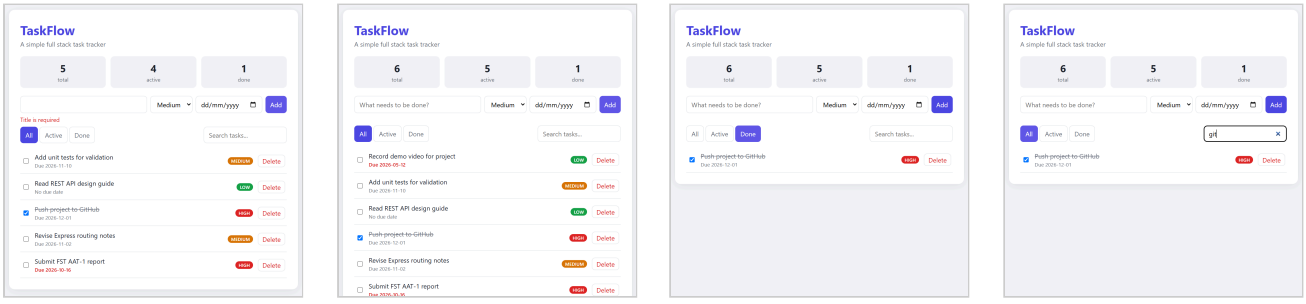
Figure 7.1: TaskFlow Home Screen showing Total / Active / Done Counters, Priority Badges, a Completed Task and an Overdue Task

7.1 Functional Analysis of the Main Screen (Figure 7.1)

- **Statistics Cards (Top):** Show **5** total, **4** active and **1** done task, fetched from `/api/stats` after every change.
- **Add-Task Form:** Accepts a title, a priority (Low / Medium / High) and an optional due date; the `Add` button posts the data to the API.
- **Task List:** Each row has a completion checkbox, a colour-coded priority badge (green / amber / red) and a `Delete` button; completed tasks are struck through.
- **Overdue Highlight:** The task “Submit FST AAT-1 report” (due 2026-10-16) is shown in bold red because it is not completed and its due date has passed.

7.2 Validation, Add, Filter and Search (Figure 7.2)

Figure 7.2 demonstrates the interactive behaviour: the server rejects an empty title with a readable error, a new task is added and appears in the list, the *Done* tab shows only completed tasks, and the search box narrows the list to matching titles.



(a) Empty title rejected: “Title is required”

(b) New task added (total becomes 6)

(c) “Done” filter shows completed tasks only

(d) Search “git” filters by title

Figure 7.2: Validation Error, Task Creation, Status Filtering and Title Search in TaskFlow

8. GITHUB REPOSITORY SCREENSHOT AND LINK

Repository Name & Visibility	**FST_AAT1** (Public Portfolio Repository)
GitHub Owner	**Veins19** (Tharun Sivakumar) — `main` branch
Direct Repository Link	https://github.com/Veins19/FST_AAT1
Uploaded Project Modules	`app.js`, `server.js`, `public/`, `test/`, `docs/screenshots/`, `package.json` and `README.md`

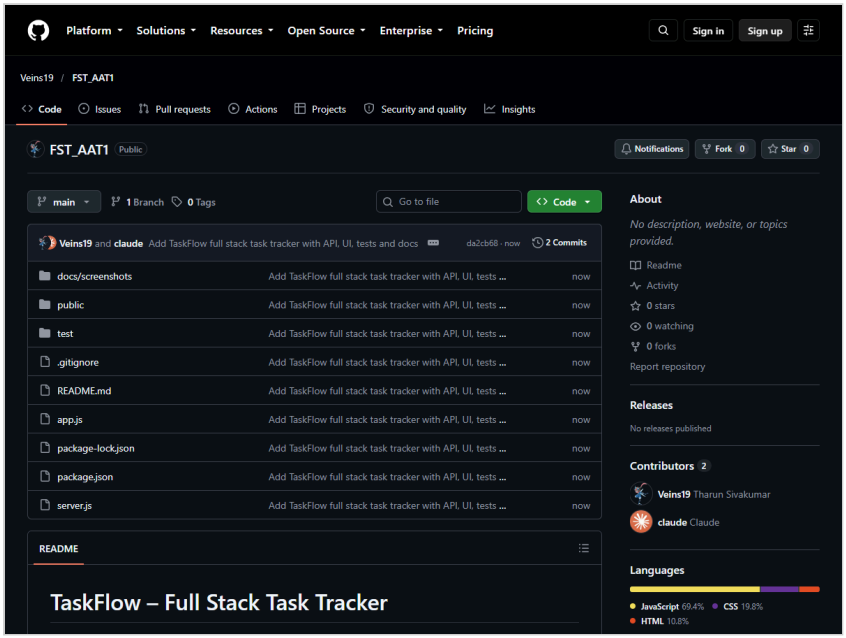


Figure 8.1: Official GitHub Repository (Veins19/FST_AAT1) showing the Source Files, Test and Public Directories, Screenshots Folder and README.md

9. CHALLENGES FACED

- **Designing a Testable Backend:** A hard-wired storage path would make tests modify real user data. The data file location was therefore passed into `createApp(dataFile)`, allowing the tests to run against a temporary file.
- **Consistent Validation for Create and Partial Update:** `POST` requires a title whereas `PUT` may send only `done`. A single `validate(body, partial)` function with a partial mode was written to serve both routes without duplicated rules.
- **Preventing Cross-Site Scripting:** Task titles are user-controlled and were first inserted through `innerHTML`. An `esc()` helper that converts `& < > " '` was added so that stored text is always displayed as plain text.
- **Test Runner and Repository Hygiene:** Node 22 failed to run `node --test test/` on a directory, so the script was changed to the explicit test file path. The runtime file `data/tasks.json` was added to `.gitignore` so that local task data is never published.

10. LEARNING OUTCOMES AND CONCLUSION

10.1 Learning Outcomes

- **Full-Stack Application Design:** Gained hands-on experience in building a REST backend with Express and connecting it to a browser frontend through the Fetch API.
- **API Design & Validation:** Learned to choose correct HTTP methods and status codes (201, 204, 400, 404) and to validate and normalise input on the server.
- **Secure Front-End Practices:** Understood why user-supplied text must be escaped before it is rendered, and how to prevent XSS in a vanilla JavaScript client.
- **Automated Testing:** Practised writing integration tests with `node:test` that start the server on a random port and use an isolated data file.
- **Git & GitHub Portfolio Management:** Learned to organise a repository, ignore runtime data, commit meaningful changes and document the project with a complete `README.md`.

10.2 Conclusion

The **TaskFlow** application successfully fulfils all requirements of the AAT-1 Portfolio-Driven Assessment for the course *Full Stack Technologies (22CSE71)*. By combining a validated Express REST API, JSON persistence and a responsive JavaScript interface with filtering, search, priorities, overdue highlighting and live statistics, the project helps users plan and track their work effectively. It is verified by automated tests and hosted and documented on GitHub under `Veins19/FST_AAT1`. Future enhancements include user authentication, a database such as MongoDB, and in-place task editing.

Signature of the Student	**Signature of the Faculty Evaluator**
Tharun Sivakumar (USN: 1NH23CS278)	Full Stack Technologies (22CSE71)